texwright

Technical documents, from an agent's first draft to the finished page

Sophia Xe5 Thomas Garren

Version 0.3.4

1 A document engine for AI workflows

Write Markdown, LaTeX, or a structured JSON document. texwright resolves its equations, references, citations, figures, and footnotes, then writes a PDF or self-contained HTML file using its own typesetting engine.

The same program checks the draft, suggests precise edits, and draws page previews. An agent can inspect both the source and the finished layout.

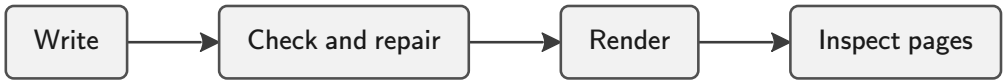


Figure 1: The document workflow, rendered by texwright.

1.1 Three ways to use it

Command line. Run checks, render files, or watch a document while editing.

MCP server. Give an agent nine tools for checking, repairing, rendering, inspecting, and creating documents.

JavaScript library. Call the conversion API from a Node.js application.

> **Note**
>
> Node.js 20 or later is required. Rendering needs no TeX installation, browser, or runtime npm dependencies. Building from source uses development dependencies.

2 What the engine handles

Table 1: Capabilities available in this source version.

Document need	texwright support
Technical writing	Sections, theorem environments, proofs, footnotes
Mathematics	TeX-style math layout, aligned equations, matrices
Navigation	Numbered objects, cross-references, page references
Research	BibTeX or CSL-JSON; numeric or author-year citations
Figures	SVG, PNG, JPEG; diagrams authored as text
Drawing languages	Mermaid, TikZ, pgfplots, forest, circuitikz, PSTricks
Page design	Three themes, brand fonts, margins, running heads
Automation	CLI, JavaScript API, stdio MCP, structured diagnostics

2.1 From meaning to pages

Readers build a shared document tree. Numbering and citation passes resolve relationships before the layout engine arranges paragraphs, mathematics, floats, and footnotes. The PDF writer embeds subset fonts; HTML carries MathML and embedded assets.

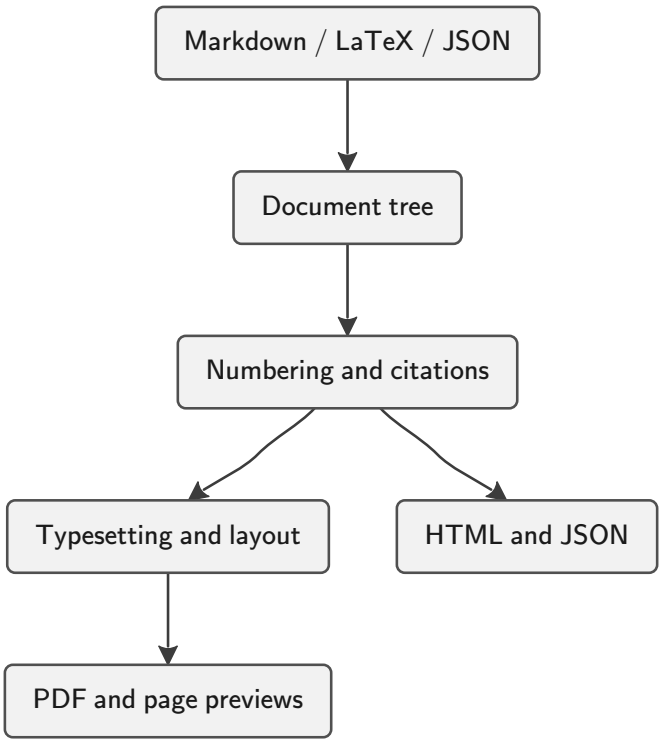


Figure 2: The shared document pipeline.

2.2 Where it fits

Use texwright for research notes, engineering reports, educational handouts, proposals, and agent-produced technical documents. Its bounded interpreters support documented subsets of drawing languages; it is not a drop-in replacement for every LaTeX package.

3 Start from this repository

Build the checked-out source with Node.js 20 or later:

```
git clone https://github.com/Chorozion/texwright.git
cd texwright
npm ci
npm run build
node dist/cli.mjs new report -o report.md
node dist/cli.mjs check report.md --diagnostics compact
node dist/cli.mjs report.md -o report.pdf
node dist/cli.mjs preview report.md --page 1
```

The repository is the installation source for this version. The unscoped npm package was not publicly available when these guides were prepared; do not assume an npm registry release exists.

3.1 Use diagnostics as the contract

A successful check verifies the document before page layout. A render can still reveal an overflowing table, delayed figure, or missing glyph. Read the exit status and diagnostics, then inspect the page image. The presence of a PDF alone does not establish success.

3.2 Known boundaries

- English hyphenation is bundled. Complex shaping and right-to-left scripts are not supported.
- PDF output is not yet tagged for screen readers.
- Remote images are not embedded in PDF. Keep SVG, PNG, or JPEG assets with the source.
- Unsupported drawing features may be approximated with diagnostics.
- Image and include access is restricted to the document folder unless explicitly widened.

3.3 Read next

Agent integration guide: MCP setup, tool selection, repair loops, and error handling.

Visual cookbook: equations, references, tables, Mermaid, and TikZ in working documents.

Full writing reference: supported syntax and diagnostic codes.

Architecture: implementation details and module boundaries.

Contributors: Sophia, Xe5, and Thomas Garren (SOPHIA XT LLC).